1. Jak korzystać z tej instrukcji

To ściąga do pokazu sklepu, krok po kroku. Najpierw coś klikasz w przeglądarce, potem pokazujesz pasujący plik. Nie musisz uczyć się nazw plików na pamięć.

Najpierw otwórz cały projekt w VS Code

1. Uruchom Visual Studio Code.
2. Kliknij **Plik > Otwórz folder** (File > Open Folder).
3. Wybierz **C:\xampp\htdocs\bytegear** i kliknij Wybierz folder.
4. Po lewej zobaczysz foldery projektu. Rozwijaj je strzałkami.

Szybszy sposób: naciśnij **Ctrl+P**, wklej pełną ścieżkę z niebieskiego pola w PDF i Enter. Potem **Ctrl+F** i wpisz podany fragment. Nie trzeba szukać konkretnego numeru linii.

Trzy rzeczy, które warto rozróżniać

Widok (.blade.php): treść strony - nagłówki, pola, przyciski. To tutaj znajdziesz HTML. Blade to HTML z dodatkami Laravel, np. wstawianiem nazwy produktu.
CSS (.css): kolory, czcionki, wielkość i rozmieszczenie elementów.
Kontroler (.php): działanie po kliknięciu, np. sprawdzenie hasła lub zapis zamówienia.

W Laravelu nie wszystko mieści się w jednym index.html. Treść strony, wspólne menu, wygląd i obsługa formularza są w osobnych plikach.

Przygotuj przeglądarkę

http://127.0.0.1:8002/

W XAMPP włącz MySQL, a sklep uruchom osobno przez Artisan. Szczegółowy start jest w pierwszym PDF-ie. Gdy strona jest nieosiągalna, samo włączenie Apache nie wystarczy.

Konta do pokazu: **customer@example.test**, **moderator@example.test**, **admin@example.test**. Wspólne hasło: **DemoSklep123!**, jeśli nie zmieniałeś go ręcznie.

Strony 2-3: mapa HTML i CSS. Strony 4-9: prezentacja zaliczenia. Strony 10-12: dodatki, proste odpowiedzi i końcowa lista kontrolna.

2. Gdzie jest HTML poszczególnych stron?

Wszystkie poniższe pliki są w folderze **resources\views**. To najszybsza mapa, gdy usłyszysz: „Proszę pokazać kod tej strony”.

`C:\xampp\htdocs\bytegear\resources\views\catalog\index.blade.php`

Ctrl+F: Małe zmiany.

Strona główna: duży nagłówek, wstęp, kategorie, wyszukiwanie i lista produktów.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\layouts\app.blade.php`

Ctrl+F: <header

Wspólna część stron: początek HTML, nazwa sklepu, menu, stopka i miejsce na komunikaty błędów.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\auth\login.blade.php`

Ctrl+F: Zaloguj się

Logowanie: pola e-mail i hasło oraz przycisk logowania.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\auth\register.blade.php`

Ctrl+F: <form

Rejestracja: pola nowego konta i potwierdzenie hasła.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\cart.blade.php`

Ctrl+F: @section

Koszyk: pozycje, liczba sztuk i podsumowanie.

Otwórz ten plik na GitHub

Dalsze pliki: **checkout.blade.php** to formularz zamówienia; **orders\index.blade.php** to historia, a **orders\show.blade.php** to szczegóły zamówienia. Wszystkie są pod resources\views.

Uwaga: welcome.blade.php to pozostawiony domyślny widok Laravel. Nie jest aktualną stroną główną sklepu.

3. Pokaż stronę główną i jej wygląd

W przeglądarce

http://127.0.0.1:8002/

Pokaż nagłówek „Małe zmiany. Większy komfort.”, menu i produkty. Wybierz kategorię, wyszukaj produkt i zmień sortowanie. To dobry początek prezentacji.

W VS Code: skąd wziął się napis na stronie?

`C:\xampp\htdocs\bytegear\resources\views\catalog\index.blade.php`

Ctrl+F: <h1>

Pokaż nagłówek pomiędzy <h1> i </h1>. Niżej <p> oznacza akapit, a <a> link. Nie musisz zmieniać pliku podczas obrony.

Otwórz ten plik na GitHub

W VS Code: gdzie są kolory i rozmiary?

`C:\xampp\htdocs\bytegear\public\css\shop.css`

Ctrl+F: .hero h1

To główny plik wyglądu sklepu. .hero h1 określa duży nagłówek, .header górne menu, a .product-grid układ produktów.

Otwórz ten plik na GitHub

W CSS **font-size** oznacza wielkość pisma, **color** kolor tekstu, **background** tło, a **padding** odstęp wewnątrz elementu. Część reguł jest zapisana w długich liniach; **Alt+Z** w VS Code włącza zawijanie tekstu.

Jak strona korzysta z CSS?

`C:\xampp\htdocs\bytegear\resources\views\layouts\app.blade.php`

Ctrl+F: asset('css/shop.css')

Linia <link rel="stylesheet" ...> dołącza CSS. Plik jest w public\css, ale w adresie zasobu nie dopisujemy public.

Otwórz ten plik na GitHub

Możesz powiedzieć własnymi słowami

„Treść strony głównej mam w widoku catalog/index.blade.php. Menu i stopka są wspólne dla stron. Wygląd ustalam w public/css/shop.css.”

4. Rejestracja, logowanie i hasła

W przeglądarce - najpierw jako osoba wylogowana

http://127.0.0.1:8002/rejestracja

1. Załóż konto, np. **pokaz15@example.test**, z imieniem Jan Pokaz.
2. Użyj hasła **PokazSklepu123!** i wpisz je ponownie w potwierdzeniu.
3. Po rejestracji zaloguj się. Przy kolejnym pokazie wybierz nowy adres, jeśli ten jest już zajęty.
4. Aktywację pokaż na następnej stronie tej instrukcji.

Gdzie jest HTML tych formularzy?

`C:\xampp\htdocs\bytegear\resources\views\auth\register.blade.php`

Ctrl+F: <form

Tu widać pola rejestracji. <input> to pole, <button> to przycisk.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\auth\login.blade.php`

Ctrl+F: type="password"

Tu jest formularz logowania. type="password" ukrywa wpisywane znaki na ekranie.

Otwórz ten plik na GitHub

A gdzie sprawdzane są dane?

`C:\xampp\htdocs\bytegear\app\Http\Controllers\AuthController.php`

Ctrl+F: public function register

register zapisuje konto; login sprawdza dane, a logout wylogowuje. validate oznacza sprawdzenie poprawności pól.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\app\Models\User.php`

Ctrl+F: 'password' => 'hashed'

To właściwe zabezpieczenie hasła w bazie. Ukrywanie znaków w polu formularza nie zastępuje hashowania.

Otwórz ten plik na GitHub

Możesz powiedzieć własnymi słowami

„Użytkownik rejestruje konto w formularzu. PHP sprawdza dane, a hasło zapisuje jako jednokierunkowy hash. Przy logowaniu Laravel sprawdza, czy podane hasło pasuje. Nie przechowuję jawnych haseł.”

5. Aktywacja e-maila i trzy role

Najpierw aktywuj nowe konto

1. Zaloguj nowe konto i otwórz „Moje konto”.
2. Otwórz http://127.0.0.1:8025/.
3. Znajdź wiadomość do swojego adresu i kliknij „Aktywuj konto”.
4. Wróć do sklepu. Ekran prośby o potwierdzenie powinien zniknąć.

`C:\xampp\htdocs\bytegear\resources\views\auth\verify-email.blade.php`

Ctrl+F: Potwierdź

Tu jest HTML ekranu oczekiwania na potwierdzenie adresu.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\app\Http\Controllers\VerificationController.php`

Ctrl+F: public function verify

Ten plik obsługuje potwierdzenie. Link jest ważny 60 minut i nie można dowolnie zmieniać jego treści.

Otwórz ten plik na GitHub

Mailpit jest lokalną skrzynką testową: odbiera prawdziwą wiadomość wysłaną przez sklep, ale nie dostarcza jej do Gmaila itp. Gdy potrzebna jest zwykła skrzynka, zmieniamy ustawienia SMTP w .env.

Następnie pokaż różne uprawnienia

http://127.0.0.1:8002/konto

Kolejno loguj trzy konta ze strony 1, wylogowując poprzednie:
Klient widzi swoje konto i zamówienia.
Moderator trafia do produktów.
Administrator trafia do użytkowników.

`C:\xampp\htdocs\bytegear\app\Http\Controllers\OrderController.php`

Ctrl+F: public function dashboard

dashboard wybiera stronę po logowaniu zależnie od roli.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\app\Http\Middleware\RequireRole.php`

Ctrl+F: 403

Ten plik odmawia dostępu osobom bez odpowiedniej roli. Jako klient wpisz ręcznie /panel/users: powinna pojawić się odmowa 403.

Otwórz ten plik na GitHub

Możesz powiedzieć własnymi słowami

„Są trzy poziomy uprawnień. Różnią się nie tylko menu: serwer też sprawdza, czy dana osoba może wejść do panelu.”

6. Administrator zarządza użytkownikami

W przeglądarce - zaloguj administratora

http://127.0.0.1:8002/panel/users

1. Pokaż listę użytkowników.
2. Wejdź w szczegóły jednej osoby.
3. Kliknij „Dodaj użytkownika” i utwórz konto testowe z nowym adresem.
4. Edytuj to konto, np. zmień imię, i pokaż wynik.
5. Usuń wyłącznie to konto testowe. Nie usuwaj kont demonstracyjnych.

Gdzie jest HTML panelu?

`C:\xampp\htdocs\bytegear\resources\views\admin\users\index.blade.php`

Ctrl+F: Użytkownicy

Lista użytkowników i przyciski. index to tutaj po prostu lista.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\admin\users\show.blade.php`

Ctrl+F: @section

Strona szczegółów jednej osoby. show oznacza pokazanie pojedynczego wpisu.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\admin\users\form.blade.php`

Ctrl+F: <form

Formularz dodawania i edycji - ten sam plik obsługuje obie sytuacje.

Otwórz ten plik na GitHub

Gdzie jest zapis i usunięcie?

`C:\xampp\htdocs\bytegear\app\Http\Controllers\AdminUserController.php`

Ctrl+F: public function store

store dodaje, update zmienia, destroy usuwa. index pobiera listę, a show jedną osobę. Hasło nadal zabezpiecza model User.

Otwórz ten plik na GitHub

Możesz powiedzieć własnymi słowami

„Panel realizuje CRUD: dodawanie, odczyt, zmianę i usuwanie. Administrator widzi wszystkich użytkowników i szczegóły jednej osoby. Aplikacja chroni przed usunięciem własnego konta administratora i ostatniego administratora.”

7. Dodawanie i usuwanie produktów

W przeglądarce - moderator lub administrator

http://127.0.0.1:8002/panel/products

1. Kliknij dodanie produktu.
2. Wpisz np. **BG Mysz Pokazowa**, kod **POKAZ-MYSZ-15**, cenę **49.90**, stan **5** i wybierz kategorię. Uzupełnij opis, pozostaw produkt aktywny.
3. Możesz dodać własne zdjęcie JPG, PNG lub WebP (do 4 MB).
4. Zapisz i pokaż produkt w sklepie.
5. Wróć do panelu, zmień cenę, zapisz, a następnie wycofaj produkt z oferty.

Gdzie jest HTML?

`C:\xampp\htdocs\bytegear\resources\views\admin\products\form.blade.php`

Ctrl+F: Dodaj produkt

Formularz nazwy, ceny, magazynu i zdjęcia. Tu pokazujesz pola produktu.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\admin\products\index.blade.php`

Ctrl+F: @section

Lista produktów dostępna dla pracownika sklepu.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\catalog\show.blade.php`

Ctrl+F: @section

Strona pojedynczego produktu, którą ogląda kupujący.

Otwórz ten plik na GitHub

Gdzie działa dodawanie i wycofanie?

`C:\xampp\htdocs\bytegear\app\Http\Controllers\AdminProductController.php`

Ctrl+F: public function destroy

store dodaje, update zmienia, destroy ustawia produkt jako nieaktywny. Pliki przesłanych zdjęć są w storage\app\private\product-images.

Otwórz ten plik na GitHub

Możesz powiedzieć własnymi słowami

„Usunięcie ze sklepu oznacza wycofanie z oferty. Rekord zostaje w bazie, żeby zachować historię zamówień. Można go później ponownie aktywować. Klient nie ma dostępu do tego panelu.”

8. Koszyk i złożenie zamówienia

W przeglądarce - klient

http://127.0.0.1:8002/koszyk

1. W sklepie dodaj dostępny produkt do koszyka.
2. Zmień liczbę sztuk i odśwież stronę: zawartość pozostaje.
3. Usuń pozycję, a potem ponownie dodaj produkt.
4. Zaloguj aktywnego klienta i przejdź do zamówienia.

`C:\xampp\htdocs\bytegear\resources\views\cart.blade.php`

Ctrl+F: @section

HTML koszyka: nazwy, ilości, przyciski i suma.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\app\Http\Controllers\CartController.php`

Ctrl+F: put('cart'

Tu zapisujemy koszyk w sesji. Sesja pamięta zawartość dla tej przeglądarki; nie trzeba zakładać konta, żeby dodać produkt.

Otwórz ten plik na GitHub

Dokończ zakup

http://127.0.0.1:8002/zamowienie

Wypełnij np. Jan Pokaz, Pokazowa 1, **00-001**, Warszawa. Kliknij „Złóż zamówienie”. To zakup demonstracyjny: pieniądze nie są pobierane. Po zapisie pokaż numer, pozycje i sumę.

`C:\xampp\htdocs\bytegear\resources\views\checkout.blade.php`

Ctrl+F: Dokończ zamówienie

HTML formularza adresu i przycisku zakupu.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\app\Http\Controllers\OrderController.php`

Ctrl+F: public function store

Tutaj powstaje zamówienie. Program sprawdza dostępność, bierze ceny z bazy, zapisuje zakup i zmniejsza magazyn.

Otwórz ten plik na GitHub

Możesz powiedzieć własnymi słowami

„Koszyk jest przechowywany w sesji. Zamówienie zapisuje się już na stałe w bazie. Cena pochodzi z bazy sklepu, więc klient nie może sobie wpisać dowolnej ceny w formularzu.”

9. Historia, status i anulowanie

W przeglądarce - klient

http://127.0.0.1:8002/zamowienia

Otwórz szczegóły właśnie złożonego zamówienia. Pokaż produkty, kwotę, adres i status. W Mailpit pokaż również wiadomość z potwierdzeniem zakupu.

`C:\xampp\htdocs\bytegear\resources\views\orders\index.blade.php`

Ctrl+F: @section

HTML historii zamówień - lista zakupów klienta.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\orders\show.blade.php`

Ctrl+F: Zamówienie #

HTML szczegółów jednego zamówienia. W tym pliku są także przyciski zmiany statusu i anulowania, widoczne dla odpowiednich osób.

Otwórz ten plik na GitHub

Wybierz dwa osobne zamówienia do pokazania

Pierwsze, jako klient: zaznacz potwierdzenie i kliknij „Anuluj zamówienie", dopóki jest nowe. Pozostaje w historii, a towar wraca do magazynu.
Drugie, jako pracownik: otwórz poniższy panel, wybierz zamówienie i rozpocznij realizację. Następnie oznacz je jako zrealizowane.

http://127.0.0.1:8002/panel/orders

`C:\xampp\htdocs\bytegear\app\Http\Controllers\OrderController.php`

Ctrl+F: public function cancel

Anulowanie sprawdza właściciela i status zamówienia. Nie można w ten sposób anulować cudzego lub już realizowanego zakupu.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\app\Http\Controllers\AdminOrderController.php`

Ctrl+F: public function update

Obsługa kolejnych statusów przez moderatora lub administratora. Lista pracownika korzysta ze wspólnego widoku orders/index.blade.php.

Otwórz ten plik na GitHub

Możesz powiedzieć własnymi słowami

„Historia przechowuje nazwę i cenę z momentu zakupu. Późniejsza zmiana ceny lub wycofanie produktu nie zmienia starego zamówienia."

10. Profil i odzyskanie hasła

Pokaż profil zalogowanego klienta

http://127.0.0.1:8002/profil

Zmień samo imię i zapisz. Opcjonalnie pokaż zmianę e-maila lub hasła. Zmiana e-maila wymaga bieżącego hasła i ponownej aktywacji. Jeśli zmienisz hasło konta demo, zapisz sobie nowe.

`C:\xampp\htdocs\bytegear\resources\views\auth\profile.blade.php`

Ctrl+F: <form

HTML edycji danych oraz zmiany hasła.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\app\Http\Controllers\ProfileController.php`

Ctrl+F: public function update

update zapisuje dane profilu, password zmienia hasło. Użytkownik nie może nadać sobie wyższych uprawnień.

Otwórz ten plik na GitHub

Pokaż „Nie pamiętam hasła” - po wylogowaniu

http://127.0.0.1:8002/zapomniane-haslo

Podaj e-mail konta testowego. Otwórz wiadomość w Mailpit, kliknij reset i ustaw nowe hasło. Zaloguj się nim. Link jest ważny 60 minut i działa jednorazowo.

`C:\xampp\htdocs\bytegear\resources\views\auth\forgot-password.blade.php`

Ctrl+F: <form

HTML formularza, w którym podaje się e-mail do odzyskania hasła.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\resources\views\auth\reset-password.blade.php`

Ctrl+F: <form

HTML ustawienia nowego hasła po kliknięciu linku z wiadomości.

Otwórz ten plik na GitHub

`C:\xampp\htdocs\bytegear\app\Http\Controllers\PasswordController.php`

Ctrl+F: public function update

Obsługa resetu. Stare zalogowane sesje są unieważniane.

Otwórz ten plik na GitHub

To dodatki do sklepu. W pierwszej kolejności pokaż wymagane funkcje ze stron 4-8 oraz zabezpieczenia ze strony 11.

11. Błędne dane i zabezpieczenia prosto

1. Formularze nie mogą przyjmować wszystkiego

Pokaż w rejestracji hasło **abc** albo niezgodne potwierdzenie. W zamówieniu wpisz kod pocztowy **123**. Popraw dane po pokazaniu błędów.

`C:\xampp\htdocs\bytegear\app\Http\Controllers\AuthController.php`

Ctrl+F: validate

required = pole wymagane; email = poprawny format; min = minimalna długość; confirmed = zgodność z potwierdzeniem. Sprawdzanie odbywa się na serwerze, nie tylko w przeglądarce.

Otwórz ten plik na GitHub

2. Wyrażenie regularne, czyli wzór poprawnego tekstu

`C:\xampp\htdocs\bytegear\app\Http\Controllers\OrderController.php`

Ctrl+F: regex:

Regex kodu pocztowego wymaga dwóch cyfr, myślnika i trzech cyfr: np. 00-001. Regex hasła w AuthController wymaga małej i wielkiej litery oraz cyfry.

Otwórz ten plik na GitHub

3. Ochrona przed SQL Injection

`C:\xampp\htdocs\bytegear\app\Http\Controllers\CatalogController.php`

Ctrl+F: where('name', 'like'

Tu jest szukanie produktu. Laravel przekazuje tekst jako wartość do wyszukania, a nie jako polecenie dla bazy. Nie sklejamy poleceń SQL z wpisanym tekstem.

Otwórz ten plik na GitHub

Możesz wpisać w wyszukiwarce **' OR 1=1 --**. To ma być potraktowane jako tekst, a nie instrukcja. Sam pusty wynik nie jest dowodem - wskaż również powyższy kod.

4. Co oznaczają te dodatki w HTML?

@csrf w formularzu dodaje zabezpieczenie przed niechcianym wysłaniem akcji z obcej strony.
{{ $zmienna }} wstawia dane do HTML tak, żeby wpisany kod HTML nie wykonał się jako część strony.

`C:\xampp\htdocs\bytegear\resources\views\layouts\app.blade.php`

Ctrl+F: $errors->any()

Tu wyświetlane są komunikaty o błędach. Ich polskie treści są w lang\pl\validation.php.

Otwórz ten plik na GitHub

Możesz powiedzieć własnymi słowami

„Sprawdzam poprawność danych przed zapisem. Stosuję wzory dla hasła i kodu pocztowego. Zapytania do bazy są parametryzowane, a formularze mają token CSRF.”

12. Ostatnie pytania i lista do zaliczenia

„Gdzie jest ustawiona baza lub nazwa sklepu?”

Otwórz **C:\xampp\htdocs\bytegear\.env**. Nazwa sklepu to **APP_NAME**, adres to **APP_URL**, a baza to **DB_DATABASE**. Ustawienia poczty zaczynają się od **MAIL_**. Po zmianie wykonaj php artisan config:clear. Nie pokazuj na nagraniu haseł ani APP_KEY; publiczny wzór to .env.example.

„Gdzie jest podłączony adres strony?”

```
C:\xampp\htdocs\bytegear\routes\web.php
```

Ctrl+F: Route::get('/'

To spis adresów. Główne / prowadzi do CatalogController i jego części index, która wybiera widok catalog.index. Nie jest to plik HTML.

Otwórz ten plik na GitHub

„Czy projekt używa API i ma testy?”

W szczegółach produktu kliknij przeliczenie ceny na EUR. Internet jest potrzebny do kursu NBP, ale nie do zakupów w PLN. Kod: **app\Http\Controllers\CatalogController.php**, wyszukaj **function euro**. HTML tej strony: **resources\views\catalog\euro.blade.php**.

Testy są w **tests\Feature**. Uruchamia je **php artisan test**. Sprawdzają wiele działań automatycznie i nie zmieniają domowej bazy. API i testy są dodatkami.

Przed końcem pokazu upewnij się, że było

1. Rejestracja, logowanie i miejsce hashowania haseł - s. 4.
2. Aktywacja wiadomością i różne role - s. 5.
3. Lista, szczegóły, dodanie, edycja i usunięcie użytkownika - s. 6.
4. Dodanie i wycofanie produktu przez pracownika - s. 7.
5. Koszyk w sesji oraz zamówienie klienta - s. 8.
6. Walidacja, regex i zabezpieczenie zapytań do bazy - s. 11.
7. HTML strony głównej, logowania oraz CSS - s. 2-4.

Technologie projektu: HTML5, CSS3, PHP 8.x, Laravel 12 i MariaDB. Można je pokazać w kodzie widoków, composer.json oraz terminalu: php --version i php artisan --version.

Nie ucz się gotowych zdań bez zrozumienia. Przećwicz pokaz, mówiąc własnymi słowami. Samodzielność, termin oddania i obrona zależą od Ciebie; ocenę przyznaje prowadzący.